UE442 - Circuits numériques et microcontrôleurs
TP3 : Processeur IP NIOS-II & Interruptions

Documents joints *: Supports ALTERA.*

Préparation

Il est demandé de consulter les documentations techniques du processeur NIOS-II, en particulier "n2cpu_nii5v1.pdf", décrivant l'architecture du processeur. Plus précisément, lire la documentation à partir de la page 3-11, décrivant les registres interne de ce processeur. L'objectif est de pouvoir utiliser les interruptions sur le NIOS-II.
Le document "DE1_Basic_Computer.pdf" explique l'utilisation des interruptions en C (détails à partir du paragraphe 3-6 page 23).

Présentation du fonctionnement des interruptions, de Qsys jusqu'à la programmation C.

Sur la base des documents de référence concernant la carte DE1, il est demandé de réponde aux questions suivantes :

1) Identifier les numéros de registres internes du processeur NIOS-II, permettant de configurer les interruptions de chaque périphérique et d'autoriser les interruptions.
2) Quelle est la fonction du registre interne ipending du processeur NIOS-II ?
3) Etudier le fonctionnement du périphérique "**Interval Timer**".
4) Lire attentivement les annexes de ce TP.

Exercice de conception

Ce travail sera réalisé sur la base d'un modèle de projet quartus (noté modèle2) et qui vous sera donné en séance.
On souhaite améliorer le programme du Chenillard du TP 1. En particulier, on souhaite gérer plus finement la temporisation de 250ms en utilisant un Timer Physique "Interval Timer" qu'on notera TIMER2, qu'on implantera au niveau de l'outil QSYS et qu'on programmera en C. Le programme principal se résumera à ceci :

```
void main () {
……..
   while(1) {
        if (Flag250ms==1 ) {
                Flag250ms=0;
                // réaliser l'affichage sur les LEDs
        }
}
```

La variable Flag250ms sera mise à 1 lors de la demande d'interruption du TIMER2 physique. Ce dernier sera configuré pour générer une interruption toutes les 250ms.
L'annexe 1 donne des détails du code d'initialisation et d'utilisation de l'interruption TIMER2.

1. Ajouter un périphérique "Interval_Timer" dans l'environnement du processeur, sous l'outil QSYS qu'on nommera TIMER2. Générer l'architecture. On choisira le numéro 4 comme numéro IRQ (Interruption).

2. Compiler l'architecture sous Quartus puis vous référer au projet C donné en exemple pour le compléter.

On souhaite utiliser les afficheurs 7 segments pour y afficher les valeurs de variables en Héxadécimal. Ces afficheurs sont interfacés au processeur NIOS-II grâce au registre HEX3_HEX0 implanté dans l'architecture sous QSYS. Ce registre est en écriture.

Dans le projet QUARTUS fourni, ce registre pilote directement les segments de chaque afficheur 7 segments, les explications de câblage sont données dans le document ***DE1_Basic_Computer.pdf***

3. Ecrire une fonction en C, notée **void Affiche7seg(unsigned short val),** permettant de transcoder la valeur de la variable **val** et de l'afficher sur les 4 afficheurs 7 segments. On favorisera fortement l'utilisation d'une LUT pour réaliser ce décodage **(attention, assurez-vous du mode de câblage des LED, anode ou cathode commune !)**.

4. Pourquoi le paramètre **val** passé à la fonction **Affiche7seg** est-il déclaré de type **short** ?

On souhaite maintenant simplifier la programmation en C de la fonction **Affiche7seg (unsigned short val)**. Pour cela, cette fonction se contentera juste d'écrire la variable **val** (passée en paramètre) dans le registre HEX3_HEX0. Le travail de transcodage vers les afficheurs 7 segments se fera au niveau matériel en VHDL.

5. Modifier la description principale en VHDL du projet QUARTUS pour implanter un module transcodeur permettant d'afficher en Hexadécimal un mot du registre HEX3_HEX0 (codé sur 16 bits).

On souhaite évaluer la performance, en temps de calcul, de l'utilisation des afficheurs 7 segments par rapport à l'utilisation de fonction **printf** sur la console de l'outil "**Monitor program**" sur l'ordinateur. **ATTENTION : dû au fait que nous contrôlons les interruptions, la fonction printf n'est plus accessible (car utilise des interruptions non identifiées) !** On va utiliser notre propre fonction **sprint** qui permet d'écrire dans une chaine de caractère (similaire à sprintf qui n'est pas accessible non plus). On utilisera ensuite la fonction standard **puts** pour afficher le texte sur la console situé sur l'ordinateur (sous l'outil Altera Monitor Program).

Les explications seront données pendant le TP.

On utilisera les fonctions de mesure du temps basées sur le TIMER, vu en TP1. On commencera par effectuer des mesures simples de la manière suivante :

```
unsigned long v1,v2, difference;
……
v1 = getsnap();
v2 = getsnap();
difference = getsnapdif(v1,v2);
//afficher la valeur de la variable difference.
```

6. Expliquer à quoi va servir ce test ? que signifie la valeur affichée ?

7. Réaliser la mesure du temps de calcul correspondant à la fonction d'affichage sur les 7 segments, dans la version logicielle et la version en VHDL, ainsi que le temps d'affichage sur la console du PC. Conclure.

On souhaite maintenant calculer le carré de la valeur correspondante aux 10 interrupteurs (SWs) codée en binaire non signé et l'afficher sur les 7 segments. Le calcul se fera par le processeur NIOS-II. Ces interrupteurs sont capturés dans le registre désigné par **Slider_switches** dans l'architecture QSYS du processeur.

D'autre part, pour ne pas submerger le programme principal, on ne réalisera ce calcul que lorsque la valeur des interrupteurs change. Il faudra modifier le registre **Slider_switches** de sorte qu'il puisse générer une interruption (à définir) sur tout changement de l'un des interrupteurs.

8. Modifier la configuration du registre de capture des interrupteurs sur l'outil QSYS, générer l'architecture puis compiler sous QUARTUS. On implantera ensuite le programme en C pour réaliser la capture sous interruption des interrupteurs SW, effectuer le calcul du carré puis afficher le résultat sur les afficheurs 7 segments. Lors de l'interruption des SW, on produira un flag à 1 qui sera alors exploitée dans le programme principal. Dès que ce flag sera positionné, on effectuera le calcul et on affichera le résultat sur l'afficheur 7 segments. Ne pas oublier de remettre à 0 ce flag.

ANNEXE 1

Exemple d'utilisation d'une interruption TIMER2

```
volatile char  flag250;
/*****************************************************/
/* Interrupt routine TIMER2                          */
/*****************************************************/
int Timer2_irq(void) {
        flag250 =1;
}

/*****************************************************/
// fonction init Timer2 et son interruption
/*****************************************************/
#define  TIMER2_BASE      ?????????????  // indiquer l'adresse de base d'accès aux registres du Timer2

void init_IT(){
        volatile int *Timer2_ptr = (volatile int *) TIMER2_BASE; // interval timer base address
        int counter = 0xBEBC20; // 1/(50 MHz) donne (0xBEBC20) = 250 msec
        *( TIMER2_BASE + 0x2) = (counter & 0xFFFF); // initialize period LSB
        *( TIMER2_BASE + 0x3) = (counter >> 16) & 0xFFFF; // initialize period MSB
        /* start interval timer, enable its interrupts */
        *( TIMER2_BASE + 1) = 0x7; // STOP = 0, START = 1, CONT = 1, ITO = 1
        /* set  mask bits interrupt  to 1 (bit 0 is Nios II reset) */
        NIOS2_WRITE_IENABLE( 0x10 ); /* set interrupt mask bits for Timer2 bit4*/
// Le numéro du bit d'interruption du périphérique est donné dans la dernière colonne sur l'outil QSYS.

// Autoriser l'interruption générale du processeur
//
        NIOS2_WRITE_STATUS( 1 ); // enable Nios-II interrupts bit PIE
}

/*****************************************************************************/
/* Macros for accessing the control registers of the NIOS-II processor.      */
/* __builtin_rdctl(x) permet l'accès aux registres interne x du CPU NIOS-II  */
/*****************************************************************************/
#define NIOS2_READ_STATUS(dest) { dest = __builtin_rdctl(0); }
#define NIOS2_WRITE_STATUS(src) { __builtin_wrctl(0, src); }
#define NIOS2_READ_ESTATUS(dest) { dest = __builtin_rdctl(1); }
#define NIOS2_READ_BSTATUS(dest) { dest = __builtin_rdctl(2); }
#define NIOS2_READ_IENABLE(dest) { dest = __builtin_rdctl(3); }
#define NIOS2_WRITE_IENABLE(src) { __builtin_wrctl(3, src); }
#define NIOS2_READ_IPENDING(dest){ dest = __builtin_rdctl(4); }
#define NIOS2_READ_CPUID(dest)  { dest = __builtin_rdctl(5); }
```

STATUS REGISTER (internal Address 0, is R/W) :

Bit Fields															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								RSIE	NMI	PRS					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRS						IL						IH	EH	U	PIE